

STREAMING JOINT SPEECH RECOGNITION AND DISFLUENCY DETECTION

Hayato Futami¹, Emiru Tsunoo¹, Kentaro Shibata¹, Yosuke Kashiwagi¹, Takao Okuda¹,
Siddhant Arora², Shinji Watanabe²

¹Sony Group Corporation, Japan ²Carnegie Mellon University, USA

ABSTRACT

Disfluency detection has mainly been solved in a pipeline approach, as post-processing of speech recognition. In this study, we propose Transformer-based encoder-decoder models that jointly solve speech recognition and disfluency detection, which work in a streaming manner. Compared to pipeline approaches, the joint models can leverage acoustic information that makes disfluency detection robust to recognition errors and provide non-verbal clues. Moreover, joint modeling results in low-latency and lightweight inference. We investigate two joint model variants for streaming disfluency detection: a transcript-enriched model and a multi-task model. The transcript-enriched model is trained on text with special tags indicating the starting and ending points of the disfluent part. However, it has problems with latency and standard language model adaptation, which arise from the additional disfluency tags. We propose a multi-task model to solve such problems, which has two output layers at the Transformer decoder; one for speech recognition and the other for disfluency detection. It is modeled to be conditioned on the currently recognized token with an additional token-dependency mechanism. We show that the proposed joint models outperformed a BERT-based pipeline approach in both accuracy and latency, on both the Switchboard and the corpus of spontaneous Japanese.

Index Terms— speech recognition, disfluency detection, streaming, Transformer

1. INTRODUCTION

Disfluencies are interruptions in the normal flow of spontaneous speech that do not appear in written text.

Flights [from Boston + {uh I mean} to Denver]

The above example shows a disfluency structure introduced in [1]. A reparandum “from Boston” is a part to be repaired. An optional interregnum “uh I mean” (such as filled pauses and discourse markers) follows after an interruption point marked as “+.” Then, a repair part “to Denver” follows. Both reparanda and interregna are regarded as disfluencies in this study.

Automatic speech recognition (ASR) systems usually transcribe such disfluencies as they are, which degrades the human readability of transcripts. They can also adversely affect downstream natural language processing (NLP) tasks, such as machine translation [2], question answering [3], and intent classification [4]. To address these issues, many studies have been conducted to detect and remove disfluencies. Most existing studies have adopted a pipeline approach, where an ASR model converts speech into text, and a disfluency detection model then tags tokens in this text. As in many NLP tasks, BERT [5] has become a popular choice for disfluency detection [6, 7]. Such pipeline approaches usually work offline, assuming complete input text. As ASR is often conducted in online streaming processing, a streaming disfluency detection method is

desirable, which works immediately after streaming ASR outputs a partial result. Recently, some studies have applied BERT to partial text incrementally [8, 9].

In this study, we consider a streaming joint model for ASR and disfluency detection. Among some options for streaming ASR modeling, we adopt a Transformer-based model because it is compatible with a token-level NLP task. The joint model utilizes acoustic features, which makes the model robust against ASR errors. In addition, non-verbal clues for disfluency detection can be used, such as interruption points [10]. Finally, the model also has the advantages of low latency and a small memory footprint.

To jointly solve ASR and disfluency detection, a transcript-enriched model can be considered. The transcript-enriched model is trained on enriched transcripts that have disfluency annotations, where special symbols for the starting and ending points of the disfluent part are added. This style of modeling is popular for solving some spoken language understanding (SLU) tasks [11, 12], and similar non-streaming variants have already been explored in non-streaming disfluency detection [13]. However, the latency will increase because of the increased sequence length with special symbols. In addition, conventional language models (LMs) cannot be straightforwardly applied, because we have to handle these special symbols included in their outputs. To overcome these shortcomings, we propose a multi-task model that has an additional output layer to streaming Transformer ASR to predict whether the next token is disfluent or not. We use disfluency detection history along with transcript history as input. We also add a novel token-dependency mechanism, to model dependencies between token and disfluency predictions. We experimentally confirmed the effectiveness of these joint models on both Switchboard and the corpus of spontaneous Japanese (CSJ) that have disfluency annotations.

2. RELATED WORK

2.1. Streaming end-to-end ASR

In streaming end-to-end ASR, both its encoder and decoder should operate online. On the encoder side, particularly for a Transformer encoder, blockwise processing has been shown to be effective, where the encoder processes fixed-length speech blocks at certain time intervals. Some studies have proposed contextual block processing [14, 15] to inherit the global context information.

On the decoder side, there are two major streams: frame-synchronous and label-synchronous models. Frame-synchronous models, such as RNN-T [16], provide frame-level predictions. Label-synchronous models, such as attention-based decoder [17, 18], provide token-level predictions. As disfluency detection is a token-level tagging task, a label-synchronous model with a Transformer decoder will be natural to solve it. Label-synchronous models need to detect the decoding end of the current block, and several ways for that have been proposed [17, 18, 19]. In this study, we

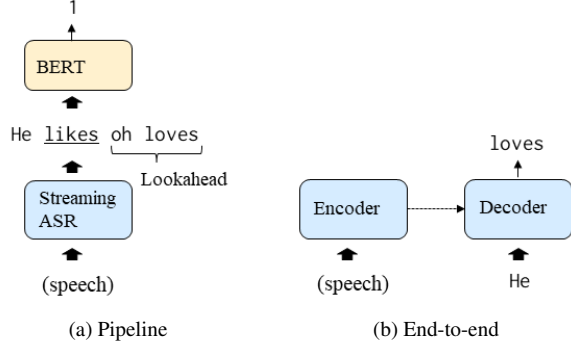


Fig. 1: (a) Pipeline method with BERT for streaming disfluency detection and (b) end-to-end disfluency removal models. The example text is “He likes oh loves her” where “oh” and “loves” are disfluencies (labeled as 1).

adopt [17] because it performs comparably without any additional training.

2.2. Disfluency detection

Disfluency detection has been mainly conducted offline and solved as post-processing of ASR, which we refer to as a “pipeline” method. Prior to Transformer, it has been solved as a sequence labeling problem using RNNs (LSTMs) and their variants [20, 21, 22]. Since the rise of Transformer, it has started to be solved as a translation task using encoder-decoder models [23, 24]. As in many NLP tasks, the most successful approach now is to fine-tune pre-trained models such as BERT [6, 7, 22].

Recently, some studies have addressed streaming, or incremental, disfluency detection [8, 9], where detection is conducted on incomplete text, as early as possible after being output from streaming ASR. To solve the input mismatch between training and inference, BERT has been trained not only on complete text but also on prefixes of them. In addition, a lookahead window, whose size is fixed or adaptive [9], plays an important role in accurate detection. The streaming disfluency detection with BERT is shown in Fig. 1 (a).

Disfluency detection has been jointly solved with dependency parsing [6] or other NLP tasks [25]. It has been solved with ASR [13, 26], where CTC or mask CTC was trained on enriched transcripts with special tags. However, streaming processing has not been considered in these studies.

Besides disfluency detection, some studies have investigated end-to-end disfluency removal, which directly predicts transcripts without disfluencies from speech [27, 28], as shown in Fig. 1 (b). From a practical point of view, such models are occasionally inapplicable, because they cannot provide transcripts for the disfluent part. Disfluencies carry some speakers’ emotions, such as a lack of confidence, which are useful for certain applications. Moreover, when an excessive removal occurs, the corresponding transcript will not be available by any means, while available by controlling a threshold in detection methods.

3. JOINT MODELING OF STREAMING SPEECH RECOGNITION AND DISFLUENCY DETECTION

In this study, we solve streaming disfluency detection by joint modeling with ASR. Compared to pipeline methods, a streaming joint model can process both tasks in a single model, leading to latency and model size reduction. It can also directly access speech information for disfluency detection. Speech helps the model deal with

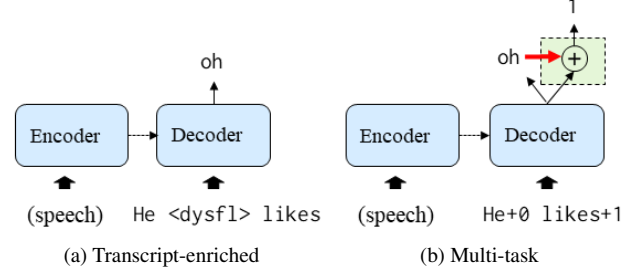


Fig. 2: Two variants of joint ASR and disfluency detection models. (a) Transcript-enriched model and (b) multi-task model. Green box indicates a token-dependency mechanism.

recognized text with ASR errors. In addition, speech carries extra information that is helpful for detecting disfluencies, which cannot be represented in text form. Specific pauses, word durations, and intonation patterns are observed before or after the interruption point, as speakers realize that they have misspoken [10, 29]. Furthermore, assuming blockwise streaming processing, the model can look at some future speech context before corresponding tokens are decoded. We implement the joint model based on streaming Transformer ASR [17] and investigate two variants of joint modeling as in Fig. 2.

3.1. Transcript-enriched model

A transcript-enriched model predicts text with disfluency tags from speech, as shown in Fig. 2 (a). We introduce special symbols for the starting and the ending points of the disfluent part marked as $\langle \text{dysfl} \rangle$ and $\langle / \text{dysfl} \rangle$. Although other tagging strategies are possible [13, 26], we adopt such a strategy to minimize the increase in sequence length. Let $\mathbf{X}$ and $\mathbf{y} = (y_1, \dots, y_L)$ denote acoustic features and a corresponding token sequence of length L , respectively. y_i comes from the vocabulary $\mathcal{V}$, i.e., $y_i \in \mathcal{V}$. Let $\mathbf{y}^+ = (y_1^+, \dots, y_{L'}^+)$ denote an enriched token sequence of length $L' (\geq L)$, where $y_i^+ \in \mathcal{V}' = \mathcal{V} \cup \{\langle \text{dysfl} \rangle, \langle / \text{dysfl} \rangle\}$. The training objective is formulated as follows:

$$\mathcal{L} = - \sum_i \log p(y_i^+ | \mathbf{X}, \mathbf{y}_{<i}^+), \quad (1)$$

where $\mathbf{y}_{<i}^+$ represents $(y_1^+, \dots, y_{i-1}^+)$.

3.2. Multi-task model

In the transcript-enriched model, disfluency tags are incorporated into token sequences $\mathbf{y}$ to make $\mathbf{y}^+$. These tags increase the sequence length from L to L' , which has a negative effect on the latency. In addition, LMs are usually trained on the untagged text and do not work well with tagged outputs from the transcript-enriched model due to the vocabulary mismatch between $\mathcal{V}$ and $\mathcal{V}'$. Although LMs can be trained on tagged text, such data are obtained from disfluency-annotated spoken text, which is often limited compared to untagged text.

To avoid using disfluency tags, we propose a multi-task model, which predicts the next token y_i first, and then determines whether this token is disfluent or not, as shown in Fig. 2 (b). The decoder has two output layers for each role. Let $\mathbf{d} = (d_1, \dots, d_L)$ denote a disfluency sequence ($d_i = 1$ for disfluent tokens and $d_i = 0$ for fluent tokens) that corresponds to $\mathbf{y}$. The joint training objective is

formulated as the following factorized form:

$$\begin{aligned}\mathcal{L} &= -\sum_i \log p(y_i, d_i | \mathbf{X}, \mathbf{y}_{<i}, \mathbf{d}_{<i}) \\ &= -\sum_i \log p(y_i | \mathbf{X}, \mathbf{y}_{<i}, \mathbf{d}_{<i}) - \sum_i \log p(d_i | \mathbf{X}, \mathbf{y}_{\leq i}, \mathbf{d}_{<i}).\end{aligned}\quad (2)$$

Note that $\mathbf{y}_{\leq i} = (\mathbf{y}_{<i}, y_i)$, which naturally incorporates the *dependency* of predicting d_i given the corresponding token y_i through the above formulation based on joint probability distribution. First, to model the *dependency* between d_i and previous disfluency predictions $\mathbf{d}_{<i}$, we input $\mathbf{d}_{<i}$ along with $\mathbf{y}_{<i}$. To feed these input representations to the network, token and disfluency input embeddings are summed, similar to token and segment embeddings in BERT [5].

To capture the *dependency* from y_i to d_i in Eq. (2), we propose a token-dependency mechanism as an additional connection between the two output layers in the green box of Fig. 2 (b). Such a dependency is important because whether a token is disfluent or not is highly related to what the token is. Note that our joint distribution formulation Eq. (2) can have another solution that makes token predictions y_i depend on disfluency ones d_i , i.e., $p(y_i | \mathbf{X}, \mathbf{y}_{<i}, \mathbf{d}_{\leq i})$, which will be compared in Section 4.2. Let y_i , $E(y_i)$, and $\mathbf{s}_i$ denote the i -th (current) predicted token, its embedding, and the i -th final hidden state of the Transformer decoder, respectively. For the i -th disfluency prediction, $E(y_i)$ and $\mathbf{s}_i$ are concatenated and linearly transformed as:¹

$$p(d_i | \mathbf{X}, \mathbf{y}_{\leq i}, \mathbf{d}_{<i}) = \text{softmax}(W[E(y_i); \mathbf{s}_i] + b), \quad (3)$$

where W and b denotes the weight and bias of the linear layer, respectively. During inference, we apply beam search with disfluency scores. We expand hypotheses as the combinations of $\mathbf{y}_{\leq i}$ and $\mathbf{d}_{\leq i}$ based on the following score:

$$\begin{aligned}\text{Score}(\mathbf{y}_{\leq i}, \mathbf{d}_{\leq i}) &= -\sum_{j=1}^i \log p(y_j | \mathbf{X}, \mathbf{y}_{<j}, \mathbf{d}_{<j}) \\ &\quad -\alpha \sum_{j=1}^i \log p(d_j | \mathbf{X}, \mathbf{y}_{\leq j}, \mathbf{d}_{<j}),\end{aligned}\quad (4)$$

where α is a hyperparameter. We found $\alpha = 1.0$ in the experiments.

4. EXPERIMENTAL EVALUATIONS

4.1. Experimental conditions

We conducted ASR and disfluency detection experiments on Switchboard 1 Release 2 (Switchboard) [30] and the CSJ [31]. Switchboard consists of 260 hours of approximately 2,400 English telephone conversations [30]. As a part of the Penn Treebank Release 3 dataset, 1,126 conversations were annotated for disfluencies [32]. Since the original transcript had some errors, Mississippi State University released corrected transcripts with utterance segmentation, which was necessary for ASR. However, disfluency annotation did not exist in the corrected “Msstate” version. Paired speech and text data with disfluency annotation were required to train joint ASR and disfluency detection models. A previous study had labeled Msstate transcripts using a state-of-the-art disfluency detector [27]. We believed that such labeling was too biased toward the detector. We instead aligned disfluency-annotated original transcripts to segmented Msstate transcripts by backtracking the path of the smallest edit distance². As a result, we obtained about 240k utterances (of 580k

utterances in the Msstate version) after speed perturbation [33]. We used the same data splits as used in most previous studies: `sw[23]*` for training data, `sw4[5-9]*` for dev data, and `sw4[1-4]*` for test data. Both *reparanda* and *interregna* were seen as disfluencies to be removed (14% of total tokens for training data), following [9]. CSJ consists of 520 hours of Japanese presentations [31], which has disfluency annotations as `TagFiller`, `TagDisfluency`, and `TagDisfluency2` (8.1% of total tokens).

We built a streaming Transformer model for joint ASR and disfluency detection, which consisted of a 12-layer encoder and a 6-layer decoder using ESPnet [34]. The encoder was based on contextual block processing [14] with 1,600 ms block size and 640 ms shift. The model was jointly trained and decoded with CTC of weight 0.3. During decoding, beam search of width 5 was applied. We also built an end-to-end disfluency removal model [27] in Fig. 1 (b), with the same architecture. ASR was done in a wordpiece unit of size 30,522 on Switchboard and a character unit of size 3,261 on CSJ.

As a pipeline approach in Fig. 1 (a), we prepared BERT with “Transformers” library³, `bert-base-uncased` for Switchboard and `cl-tohoku/bert-base-japanese-char-v2` for CSJ. We fine-tuned them on text with disfluency labels. For streaming disfluency detection, we fine-tuned BERT on all the prefixes [7, 9], as noted in Section 2.2. For ASR on Switchboard, we use the same tokenizer as BERT to make a fair comparison.

We evaluated disfluency detection with ASR in terms of both accuracy and latency. As an accuracy metric, we defined the disfluency removed word error rate (DR-WER). We calculated WER on only fluent words, where hypotheses after removing disfluencies were compared with references without disfluencies. While most previous studies in this field have reported F1 scores on the reference transcripts, we reported the performance on ASR results, where F1 scores could not be calculated straightforwardly. We calculated F1 scores by aligning hypotheses with the references as “aligned” F1 score⁴. However, it was affected by alignment mismatch, and it was difficult to say which was a disfluent part in the erroneously recognized text. Therefore, we regards the DR-WER as the target metric for accuracy, while aligned F1 scores as an auxiliary metric. As a latency metric, we calculated the token-level latency, i.e., the elapsed time from the acoustic boundary of a token to disfluency detection of the token, as noted in [35]. The boundary was obtained from CTC segmentation [36]. Latency calculation was conducted in teacher-forced decoding (beam width was 1) [35].

4.2. Experimental results

First, we compared different models for streaming ASR and disfluency detection on Switchboard in Table 1. The first column shows the original WER including disfluencies. We found that both joint models did not hurt and even improved the WER, compared to the ASR-only model. The end-to-end removal model output only fluent transcripts, so the WER including disfluencies could not be calculated. Note that the WER became higher compared to the standard Switchboard setup with the Msstate transcripts, because of inaccurate original transcripts and smaller amounts of training data, as noted in Section 4.1. The second column shows the DR-WER. For the ASR-only model, all the disfluencies remained and were counted as errors. We found that both joint models outperformed the pipeline approaches, where looking ahead improved the DR-WER at the cost of the latency but still fell short of joint models. Our joint models also outperformed the end-to-end removal model, with flexibility to

¹For simplicity, this formulation omits complicated notation of streaming processing. y_i and d_i are actually predicted before the entire $\mathbf{X}$ are obtained.

²We will release script: <https://github.com/hayato-futami-s/joint-asr-dysfl>

³<https://huggingface.co/models>

⁴Token inserted or deleted parts do not contribute to the F1 calculation.

Table 1: Streaming ASR and disfluency detection on Switchboard without LM. “WER” denotes the word error rate including disfluencies, which is a standard ASR metric. “DR-WER” denotes disfluency removed WER. “Aligned F1 / P / R” denotes F1, precision, and recall score calculated by aligning ASR results to references. “TL-Latency” denotes the 50th/90th percentile of token-level latency.

	WER[%]	DR-WER[%]	Aligned F1 / P / R	TL-Latency (50/90) [ms]	Parameters
ASR only [17]	18.1	36.9	—	496/978	50.6M
Joint (Multi-task)	17.8	21.3	0.84 /0.89/0.79	529/995	58.4M
Joint (Transcript-enriched)	17.9	21.4	0.84 /0.89/0.80	575/1061	50.6M
End-to-end removal [27]	—	21.7	(0.78/0.78/0.78)	482/860	50.6M
Pipeline ASR + BERT (No Lookahead)	18.1	26.5	0.69/0.80/0.60	536/1045	159M
Pipeline ASR + BERT (Lookahead=2)	18.1	22.6	0.81/0.86/0.76	641/1199	159M

Table 2: Ablation studies on components for multi-task model

	DR-WER[%]
Multi-task	21.3
- token-dependency mechanism	22.3
- input disfluency	22.7
Disfluency-dependency mechanism	22.2

choose fluent or disfluent hypotheses. For the type of joint modeling, the multi-task model was comparable to the transcript-enriched model. The third column shows aligned F1, precision (P), and recall (R) scores. Similar trends to the DR-WER were observed. Note that these values could not be directly compared to previous studies due to ASR error handling and utterance segmentation. For the end-to-end model, ASR deletion errors and detection errors cannot be distinguished, which may not be a fair comparison to others. The fourth column shows the 50th and 90th percentile of the token-level latency calculated on a Intel(R) Core(TM) i7-12700KF CPU. The pipeline required one more step for BERT inference and waited for future tokens to achieve sufficient accuracy. As a result, both joint models achieved lower latency than BERT with a 2-token lookahead window. Comparing the two joint models, the multi-task model was better at latency, because the output length of the transcript-enriched model was increased, as discussed in Section 3.2. The end-to-end model had the lowest latency because its output length was decreased by removing disfluencies. The final column shows the number of total parameters. In the multi-task model, the token-dependency mechanism increased the model size, mainly because of the embedding layer for 30,522 vocabularies in Eq. (3). However, both joint models were still much smaller than the BERT-based pipeline.

Next, we conducted ablation studies on the components for the proposed multi-task model in Table 2. We found that the token-dependency mechanism in Eq. (3) was necessary, as removing it degraded the DR-WER significantly, which shows the importance of the dependency of the current token y_i to predict d_i . We also found that it was important to input previous disfluency predictions besides token predictions to the model, as $d_{<i}$ in Eq. (2). Instead of the token-dependency mechanism, we added the disfluency-dependency mechanism as discussed in Section 3.2, whose performance was worse.

We evaluated the models with shallow fusion with LMs in Table 3. The training data for LMs were Switchboard transcripts and the external Fisher corpus [37]. In addition to a standard RNNLM, we prepared a transcript-enriched RNNLM, where disfluency tags were added in the part of LM training data that has disfluency annotations. Note that this requires an extra cost of training a special LM this is only applicable when in-domain disfluency-annotated data are available, as discussed in Section 3.2. In the multi-task model, the WER and DR-WER were successfully improved by using the LM. However, in the transcript-enriched model, the WER and DR-WER

Table 3: LM integration for disfluency detection models

	WER[%]	DR-WER[%]
ASR only	18.1	36.9
+ standard LM	17.0	35.0
Joint (Multi-task)	17.8	21.3
+ standard LM	16.8	20.1
Joint (Transcript-enriched)	17.9	21.4
+ standard LM	17.6	21.6
+ Transcript-enriched LM	17.3	20.1
End-to-end removal	—	21.7
+ standard LM	—	20.6

Table 4: Streaming ASR and disfluency detection on CSJ. “LA” means lookahead.

	DR-CER[%] eval1/2/3	TL-Latency eval1 50/90
ASR only	13.9/10.5/11.5	434/880
Joint (Multi-task)	5.1/3.7/4.3	449/876
Joint (Transcript-enriched)	5.1/3.8/4.2	465/886
End-to-end removal	5.2/3.8/4.2	438/810
ASR + BERT (No LA)	6.6/5.1/5.9	486/975
ASR + BERT (LA=2)	5.5/4.0/4.7	521/990

were not improved well with the standard LM due to the vocabulary mismatch based on the disfluency tags. With the transcript-enriched LM, the DR-WER was recovered.

Finally, We confirmed that the similar trends as Switchboard were observed in CSJ, as shown in Table 4. We observed that our joint models performed better than the pipeline approaches with lower latency. Also, the multi-task model achieved comparable disfluency removed CER (DR-CER) to the transcript-enriched model while achieving lower latency.

5. CONCLUSIONS

In this study, we have proposed streaming joint ASR and disfluency detection. Compared to existing pipeline approaches, joint modeling has the advantages of using acoustic information, lower latency, and a smaller memory footprint. The transcript-enriched joint model adds special symbols to the vocabulary to indicate the disfluent part, which causes latency degradation and a failure of standard LM integration. We propose a multi-task model that performs both token and disfluency predictions with two output layers. To model the token-disfluency dependency, a token-dependency mechanism is employed to bridge the two output layers. We experimentally confirmed that our joint models outperformed pipeline methods and that the multi-task model achieved competitive accuracy to the transcript-enriched model with lower latency. In a future study, we will investigate leveraging data without disfluency annotations.

6. REFERENCES

- [1] Elizabeth Shriberg, "Preliminaries to a theory of speech disfluencies," Tech. Rep., 1994.
- [2] Elizabeth Salesky, Susanne Burger, Jan Niehues, and Alex Waibel, "Towards fluent translations from disfluent speech," in *SLT*, 2018.
- [3] Aditya Gupta, Jiacheng Xu, Shyam Upadhyay, Diyi Yang, and Manaal Faruqi, "Disfl-QA: A benchmark dataset for understanding disfluencies in question answering," in *ACL-IJCNLP*, 2021, pp. 3309–3319.
- [4] Mai Hoang Dao, Thinh Truong, and Dat Quoc Nguyen, "From disfluency detection to intent detection and slot filling," in *Interspeech*, 2022, pp. 1106–1110.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL*, 2019, pp. 4171–4186.
- [6] Paria Jamshid Lou and Mark Johnson, "Improving disfluency detection by self-training a self-attentive model," in *ACL*, 2020, pp. 3754–3763.
- [7] Johann C. Rocholl, Vicky Zayats, Daniel D. Walker, Noah B. Murad, et al., "Disfluency detection with unlabeled data and small BERT models," in *Interspeech*, 2021, pp. 766–770.
- [8] Morteza Rohanian and Julian Hough, "Best of both worlds: Making high accuracy non-incremental transformer-based disfluency detection incremental," in *IJCNLP*, 2021, pp. 3693–3703.
- [9] Angelica Chen, Vicky Zayats, Daniel Walker, and Dirk Padfield, "Teaching BERT to wait: Balancing accuracy and latency for streaming disfluency detection," in *NAACL*, 2022, pp. 827–838.
- [10] Vicky Zayats and Mari Ostendorf, "Giving attention to the unexpected: Using prosody innovations in disfluency detection," in *NAACL*, 2019, pp. 86–95.
- [11] Motoi Omachi, Yuya Fujita, Shinji Watanabe, and Matthew Wiesner, "End-to-end ASR to jointly predict transcriptions and linguistic annotations," in *NAACL*, 2021, pp. 1861–1871.
- [12] Siddhant Arora, Siddharth Dalmia, Pavel Denisov, Xuankai Chang, et al., "ESPnet-SLU: Advancing spoken language understanding through espnet," in *ICASSP*, 2022, pp. 7167–7171.
- [13] Motoi Omachi, Yuya Fujita, Shinji Watanabe, and Tianzi Wang, "Non-autoregressive end-to-end automatic speech recognition incorporating downstream natural language processing," in *ICASSP*, 2022, pp. 6772–6776.
- [14] Emiru Tsunoo, Yosuke Kashiwagi, Toshiyuki Kumakura, and Shinji Watanabe, "Transformer ASR with contextual block processing," in *ASRU*, 2019, pp. 427–433.
- [15] Yangyang Shi, Yongqiang Wang, Chunyang Wu, Ching-Feng Yeh, et al., "Emformer: Efficient memory transformer based acoustic model for low latency streaming speech recognition," in *ICASSP*, 2021, pp. 6783–6787.
- [16] Yanzhang He, Tara N. Sainath, Rohit Prabhavalkar, Ian McGraw, et al., "Streaming end-to-end speech recognition for mobile devices," in *ICASSP*, 2019, pp. 6381–6385.
- [17] Emiru Tsunoo, Yosuke Kashiwagi, and Shinji Watanabe, "Streaming transformer ASR with blockwise synchronous beam search," in *SLT*, 2021, pp. 22–29.
- [18] Chung-Cheng Chiu and Colin Raffel, "Monotonic chunkwise attention," in *ICLR*, 2018.
- [19] Niko Moritz, Takaaki Hori, and Jonathan Le Roux, "Triggered attention for end-to-end speech recognition," in *ICASSP*, 2019, pp. 5666–5670.
- [20] Shaolei Wang, Wanxiang Che, Yue Zhang, Meishan Zhang, and Ting Liu, "Transition-based disfluency detection using LSTMs," in *EMNLP*, 2017, pp. 2785–2794.
- [21] Paria Jamshid Lou, Peter Anderson, and Mark Johnson, "Disfluency detection using auto-correlational neural networks," in *EMNLP*, 2018, pp. 4610–4619.
- [22] Nguyen Bach and Fei Huang, "Noisy BiLSTM-based models for disfluency detection," in *Interspeech*, 2019.
- [23] Feng Wang, Wei Chen, Zhen Yang, Qianqian Dong, et al., "Semi-supervised disfluency detection," in *COLING*, 2018, pp. 3529–3538.
- [24] Qianqian Dong, Feng Wang, Zhen-Le Yang, Wei Chen, et al., "Adapting translation models for transcript disfluency detection," in *AAAI*, 2019.
- [25] Dongyub Lee, Byeongil Ko, Myeong Cheol Shin, Taesun Whang, et al., "Auxiliary sequence labeling tasks for disfluency detection," in *Interspeech*, 2021, pp. 4229–4233.
- [26] Hirofumi Inaguma, Masato Mimura, Koji Inoue, Kazuyoshi Yoshii, and Tatsuya Kawahara, "An end-to-end approach to joint social signal detection and automatic speech recognition," in *ICASSP*, 2018, pp. 6214–6218.
- [27] Paria Jamshid Lou and Mark Johnson, "End-to-end speech recognition and disfluency removal," in *EMNLP*, 2020, pp. 2051–2061.
- [28] Koharu Horii, Meiko Fukuda, Kengo Ohta, Ryota Nishimura, et al., "End-to-end spontaneous speech recognition using disfluency labeling," in *Interspeech*, 2022, pp. 4108–4112.
- [29] Elizabeth Shriberg, "Phonetic consequences of speech disfluency," in *ICPhS*, 1999, pp. 619–622.
- [30] J.J. Godfrey, E.C. Holliman, and J. McDaniel, "SWITCHBOARD: telephone speech corpus for research and development," in *ICASSP*, 1992.
- [31] K. Maekawa, "Corpus of Spontaneous Japanese : its design and evaluation," *SSPR*, 2003.
- [32] Mitchell P. Marcus, "Treebank-3 LDC99T42," *Linguistic Data Consortium*, 1999.
- [33] Tom Ko, Vijayaditya Peddinti, Daniel Povey, and Sanjeev Khudanpur, "Audio augmentation for speech recognition," in *Interspeech*, 2015, pp. 3586–3589.
- [34] Shinji Watanabe, Takaaki Hori, Shigeki Karita, Tomoki Hayashi, et al., "ESPnet: End-to-End Speech Processing Toolkit," in *Interspeech*, 2018, pp. 2207–2211.
- [35] Hirofumi Inaguma and Tatsuya Kawahara, "Alignment knowledge distillation for online streaming attention-based speech recognition," *arXiv*, 2021.
- [36] Ludwig Kurzinger, Dominik Winkelbauer, Lujun Li, Tobias Watzel, and Gerhard Rigoll, "CTC-segmentation of large corpora for german end-to-end speech recognition," in *SPECOM*, 2020.
- [37] Christopher Cieri, David Miller, and Kevin Walker, "The Fisher Corpus: a resource for the next generations of speech-to-text," in *LREC*, 2004.